DCDS ESE PROJECT PRESENTATION

IoT Component Management System for SIT Labs (LabTrack)

Course: DCDS – ESE Project

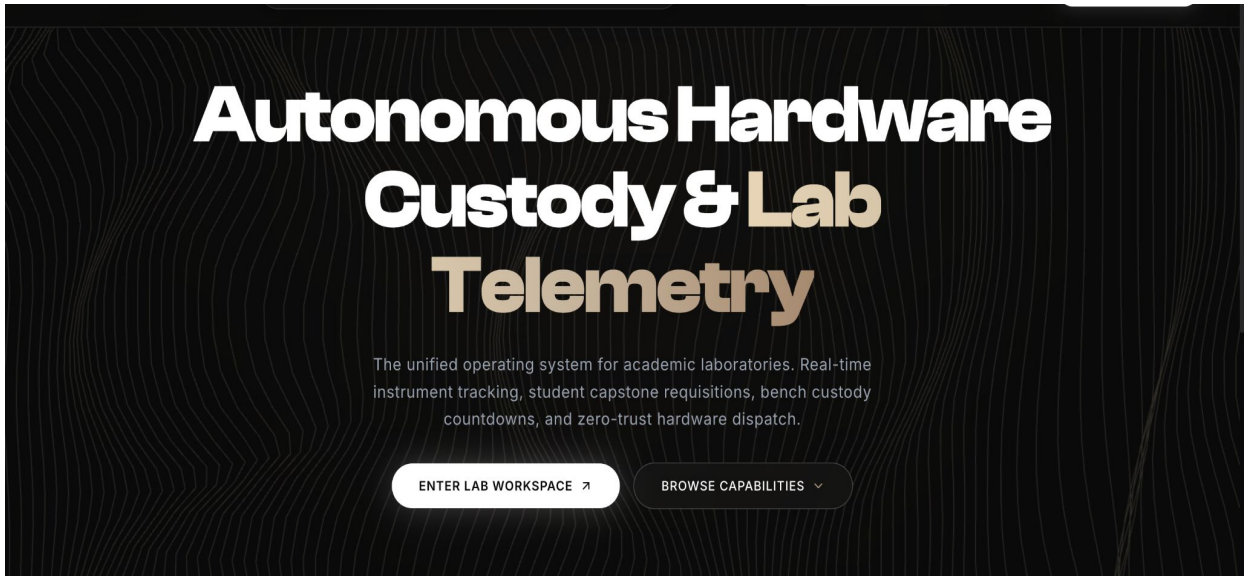
SY Division: A | Group No.: 14

Ishaan Chand 25070126076

Galidevara Raghunandana 25070126065

Ansh Arora 25070126030

Faculty Guide: Dr Anjali Dalvi



Problem Statement & Motivation

A Project-Linked Hardware Inventory and Tracking System for SIT LabsProject-Linked Hardware Inventory and Tracking System for SIT LabsProject-Linked Hardware Inventory and Tracking System for SIT Labs

The hardware exists. But who knows where it is?

Component Issued ➔ Record Missing ➔ Availability Unknown ➔ Project Delayed

Core Problem

- Unrecorded handovers → unclear custody
- Fragmented stock records → uncertain availability
- No demand history → procurement blind spots
- Lost location & custody info (not stolen hardware)

Significance

- Project teams face preventable hardware stalls
- Unrecorded circulation creates accountability gaps
- Reactive procurement without demand history
- Observed in SymbiTech's Club Events, R&A Capstones and Overall Department

Target Audience

- **Initial Scope:** SIT R&A Dept (Guidance: Manoj Kumar Sir)
- **Primary Users:** Student Project Teams & Lab Staff
- **Oversight:** Faculty Mentors & Dept Admins
- **Future Rollout:** Expansion across all SIT labs

Objectives & Scope

Project Goals, Current Capabilities and Future Boundaries

From fragmented hardware records to a traceable project lifecycle.

01 **Visibility**
Live component availability across laboratories

02 **Allocation**
Minimum-lab hardware fulfilment planning

03 **Governance**
Role-specific academic approval workflows

04 **Accountability**
Traceable custody, issue and return

05 **Intelligence**
Unmet-demand tracking for procurement insights

CURRENT PROJECT SCOPE

- SIT R&A laboratory operations
- Components, projects and requisitions
- Approvals, custody and demand visibility

FUTURE / OUTSIDE CURRENT SCOPE

- Institution-wide SIT deployment
- Physical RFID/locker integration
- Predictive procurement optimization

**Note: Institutional rollout across SIT departments remains prospective.*

System Overview / Proposed Solution

How LabTrack Works from Hardware Request to Return

Role-Based Web Interface

React Single-Page Application

- **Student**
  Project BOM & hardware requests
- **Faculty Mentor**
  Technical & project review
- **HoD**
  Final serialized asset sign-off
- **Lab Assistant**
  Physical issue & return inspection

Live Catalogue · Multi-Lab Dispatch Tracking

Python FastAPI Application & Lifecycle Services

Core Backend Engine & Business Logic Orchestration

01 · Inventory & Allocation

Live stock across labs · Deterministic minimum-lab fulfillment planning

02 · Approvals & Custody

Conditional multi-stage academic routing · Atomic physical issue & return reconciliation

03 · Events & Demand Intelligence

Immutable state-transition audit logs · Unmet-demand capture & notification triggers

Cross-Cutting Controls: Server-Side RBAC · Whole-Request Atomicity · Rollback Safety

Aiven MySQL: System of Record

Authoritative Relational State (ACID)

- Class-specific inventory & barcoded asset state
- Projects, multi-stage approvals & custody ledger
- Immutable audit history & transactional SQL outbox

Background Event Worker (Reads Outbox ⇢ Projections)

MongoDB Atlas: Document & Analytics Store

Flexible Documents & Read Projections

- Flexible BSON component specification documents
- User notification queues & delivery projections
- Aggregated unmet-demand analytics for procurement

COMPLETE HARDWARE LIFECYCLE

Every authorized state transition is recorded.

1. **Request** — Project BOM
2. **Allocate** — Min-Lab Plan
3. **Approve** — Mentor & HoD
4. **Issue** — Atomic Custody
5. **Return & Reconcile** — Condition & Audit

Requirements & Technology Stack

Technologies Used to Build and Run LabTrack

• HOST RUNTIME	• CLIENT PLATFORM	• DATA INGESTION	• TEST HARNESS
Python 3.11 on Uvicorn ASGI High-concurrency event loop	**Modern Browser (HTTPS)** Zero client installation	**openpyxl Multi-Sheet ETL** Direct Excel workbook ingestion	**pytest Automated Suite** Regression & policy validation

1. CLIENT LAYER (REACT 18 + VITE)

- **Reactive single-page catalog UI**
- Instant component search & filter
- Browser session requisition state

2. API & POLICY ENGINE (FASTAPI)

- **Pydantic V2 typed request validation**
- Server-side multi-tier RBAC rules
- Transactional Outbox event emitter

3. DATA INGESTION & TEST (ETL)

- **openpyxl workbook parser & mapper**
- pytest policy & boundary tests
- Automated rollback regression tests

AIVEN MYSQL (SYSTEM OF RECORD) CLOUD ACTIVE: AIVEN SERVER

Driver & ORM: SQLAlchemy + PyMySQL Pooling

- **Relational Integrity:** Strict PK/FK table graph
- **Concurrency Control:** ACID transaction locks
- **Atomic Counters:** Guarantees zero inventory drift
- **Advanced SQL:** Stored procedure (lab usage rollup)

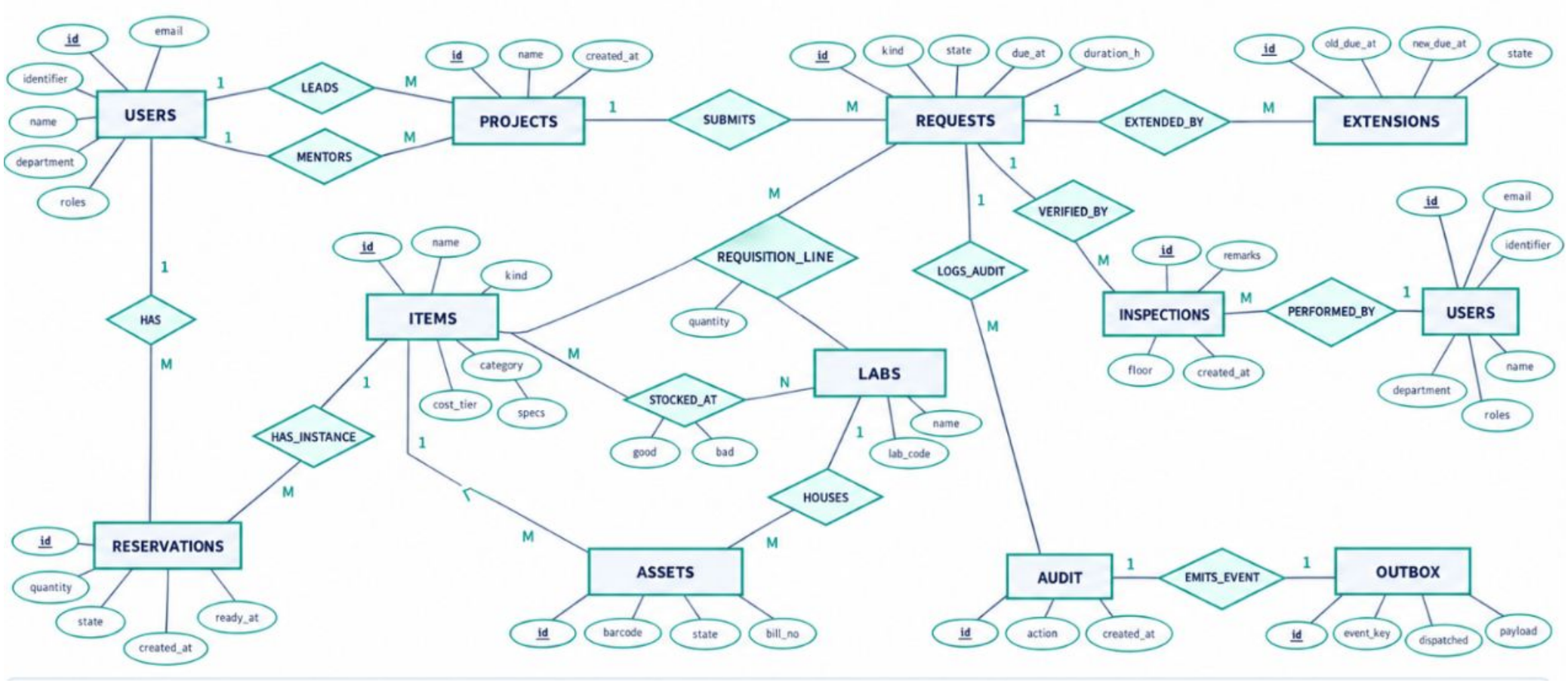
MONGODB ATLAS (DOCUMENT & ANALYTICS) CLOUD ACTIVE: ATLAS CLUSTER

Driver & Worker: PyMongo + Asynchronous Outbox Worker

- **Polymorphic Specs:** Variable pinouts & sensor datasheets
- **Outbox Consumer:** Projects events without blocking API
- **Demand Analytics:** Aggregation pipelines without JOINs
- **Schema Agnostic:** Ingests new parts without migrations

Database Design – ER Diagram

How the Main Entities and Their Relationships Are Organized



Aiven MySQL: Transactional Records | **Outbox → Worker** | **MongoDB Atlas:** Documents & Demand Analytics

Relational Schema & Normalization

Reducing Duplicate Data and Maintaining Inventory Consistency

NORMALIZATION PROGRESSION PIPELINE

0NF

0NF (Unnormalized)

Flat ledger rows with repeated student, lab, and hardware fields causing severe update/deletion anomalies.

1NF

1NF (Atomic Rows)

Split compound checkout strings into single atomic rows; introduced request_lines with discrete PKs.

2NF

2NF (Full Key Dependency)

Extracted hardware metadata into items so catalog specs no longer depend on composite loan keys.

3NF

3NF (Transitive Elimination)

Removed transitive chains (Borrower → Dept → HoD); decoupled into users, labs, and projects with strict foreign keys.

CORE SCHEMA KEY MATRIX (DATA DICTIONARY)

Table	Primary Key (PK)	Foreign Keys (FK)	Core Constraints
`users`	`id`	None	UQ(identifier, email)
`labs`	`id`	incharge_id (users)	floor, wing
`items`	`id`	None	Check: kind in (ASSET, CONSUMABLE)
`stock`	`id`	item_id, lab_id	UQ(item_id, lab_id), Check: good, bad >= 0
`assets`	`id`	item_id, lab_id	UQ(barcode), status enum
`requests`	`id`	project_id, borrower_id	state, due_at
`request_lines`	`id`	request_id, item_id, lab_id	quantity > 0

Implementation & SQL Operations

Demonstrating Relational Queries and Document-Based Analytics

01 / AIVEN MYSQL: RELATIONAL QUERY EXECUTION · Stored Procedure (labtrack_usage_by_lab) · Multi-table JOIN & Aggregation

Executable SQL: `CALL labtrack_usage_by_lab('ROB-01');`

```
-- Stored Procedure: labtrack_usage_by_lab('ROB-01')
CREATE PROCEDURE labtrack_usage_by_lab(IN p_lab VARCHAR(16))
BEGIN SELECT i.name AS item_name, i.kind, l.lab_code,
  s.good AS stock_qty, COUNT(a.id) AS issued_assets,
  SUM(CASE WHEN a.state='IN_USE' THEN 1 ELSE 0 END) AS active_qty
  FROM items i JOIN stock s ON i.id = s.item_id
  JOIN labs l ON s.lab_id = l.id
  LEFT JOIN assets a ON (a.item_id=i.id AND a.lab_id=l.id)
  WHERE l.lab_code = p_lab GROUP BY i.id, s.id, l.lab_code;
```

Returned Result: CALL labtrack_usage_by_lab('ROB-01') · 4 rows (Aiven MySQL 8.0)

item_name	kind	lab_code	stock_qty	active_assets
NVIDIA Jetson Orin Nano	ASSET	ROB-01	12 units	9 in-use
LiPo 3S Battery 5200mAh	CONSUMABLE	ROB-01	45 units	0 (batch)
Ultrasonic Sensor HC-SR04	CONSUMABLE	ROB-01	80 units	0 (batch)
TurtleBot4 Mobile Base	ASSET	ROB-01	6 units	5 in-use

02 / MONGODB ATLAS: DOCUMENT AGGREGATION · Multi-stage Aggregation Pipeline · Real-time Demand Analytics

Aggregation Pipeline: $match → $group → $project → $sort (Unmet Component Demand)

```
// MongoDB Atlas: Pipeline aggregating unfulfilled requisitions by item
db.unmet_demand_events.aggregate([
  { $match:   { 'target_lab': 'ROB-01', 'status': 'PENDING' } },
  { $group:   { _id: '$item_name',
                total_demand: { $sum: '$requested_qty' },
                team_count:   { $addToSet: '$project_id' } } },
  { $project: { component: '$_id', total_demand: 1,
                project_teams: { $size: '$team_count' }, _id: 0 } },
  { $sort:    { total_demand: -1 } }
]);
```

Aggregated Results: BSON Output · Unmet Robotics Demand (MongoDB Atlas)

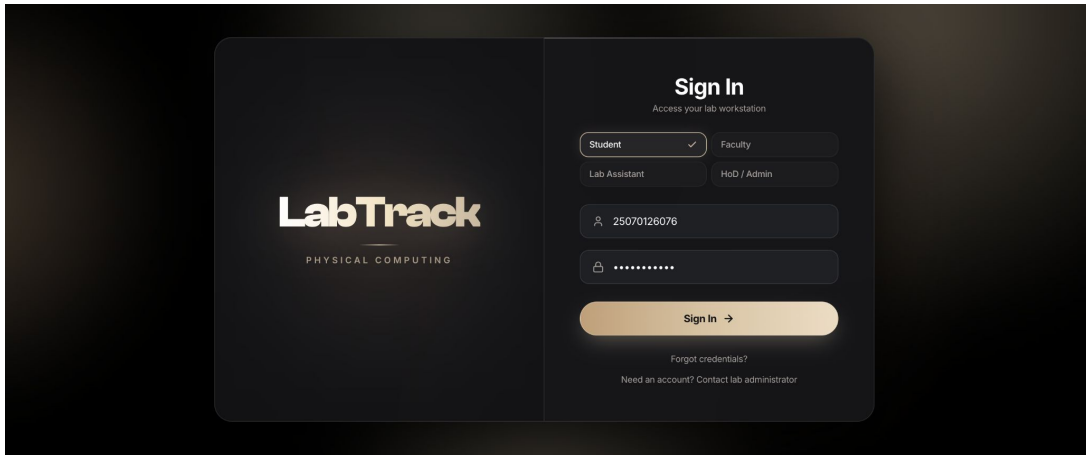
Component	Total Demand	Frequency / Teams
NVIDIA Jetson Orin Nano	18 units	7 project teams
LiPo 3S Battery 5200mAh	34 units	12 project teams
Ultrasonic Sensor HC-SR04	56 units	16 project teams

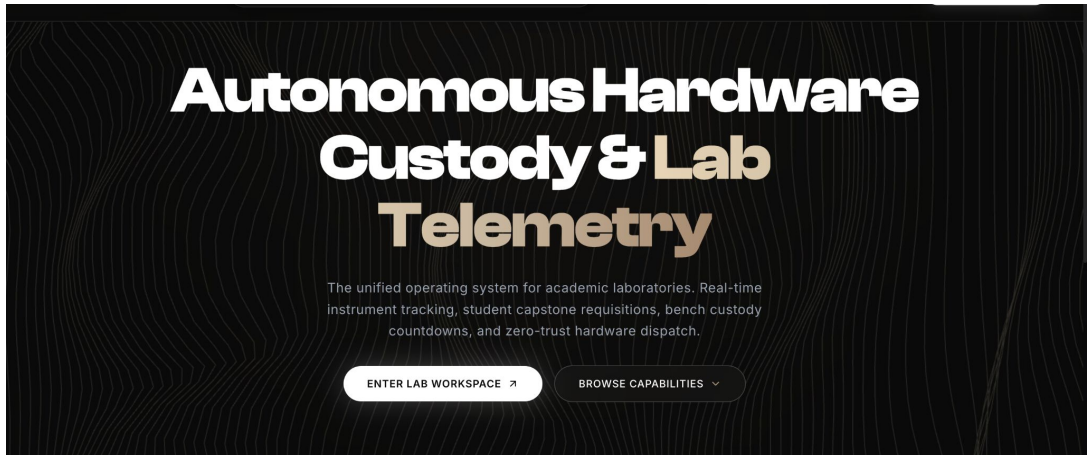
Data Provenance & Verification: Transactional records written to Aiven MySQL; transactional outbox event publisher relays requisition states to MongoDB Atlas for zero-lock polymorphic demand aggregation.

Python Integration: `SQLAlchemy` → Aiven MySQL | `PyMongo` → MongoDB Atlas

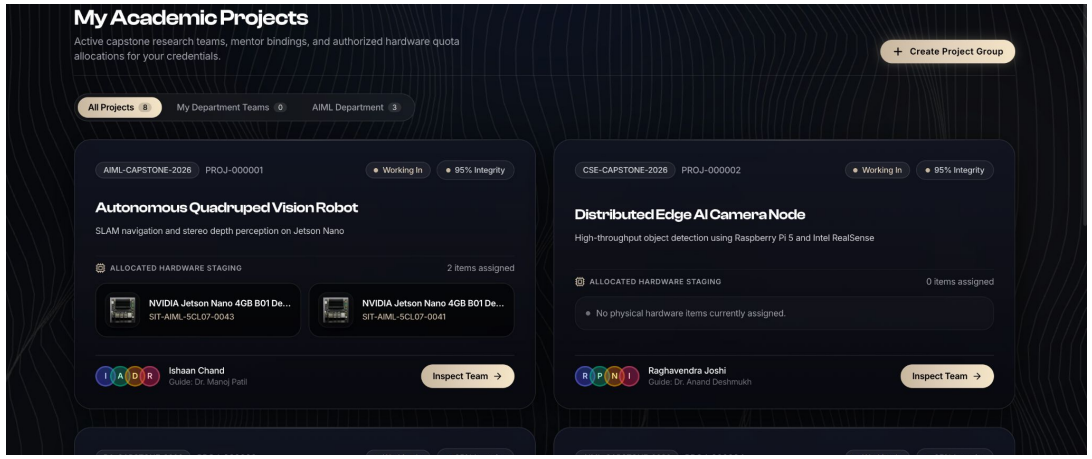
Results / Screenshots / Demonstration

The Implemented Interface and Its Practical Workflow









Testing & Validation

Verifying Security, Stock Integrity and Correct System Behaviour

Test ID	Validation Scenario	Expected Behavioral Invariant	Observed Outcome
T01 Lifecycle	Project-linked hardware requisition, faculty approval, dispatch & return.	Deterministic state progression; immutable custody audit logging; ACID consistency across entire order lifecycle.	**PASS —** 100% verified E2E lifecycle (Submit → HoD Sanction → Physical Dispatch → Custody Release)
T02 Stock Integrity	Requisition attempt exceeding available departmental inventory.	Allocation rejected; zero negative stock; SQL transaction rollback; strict check constraint (good >= 0, bad >= 0).	**PASS —** HTTP 400 rejection on over-allocation; 0 negative stock; Atomic SQL rollback verified
T03 Security & RBAC	Student attempting supervisor override or counter dispatch.	Server-side 403 Forbidden; protected custody tables unmutated; zero client-side privilege escalation.	**PASS —** HTTP 403 Forbidden strictly enforced on unprivileged roles; Zero unauthorized mutations
T04 Projection	SQL Outbox event relay processed by background worker.	Asynchronous ingestion to MongoDB; idempotent deduplication on retry; zero mutation of authoritative SQL records.	**PASS —** Idempotent Outbox deduplication verified; Zero duplicate projections on replay

PASSED: 61 Tests (100%) | **FAILED: 0 Tests** | **SKIPPED: 2 Tests (Live Cloud Opt-in)**

Execution Verification: pytest tests/ • 61 passed, 0 failed, 2 skipped in 38.22s • Python 3.13 / FastAPI / SQLAlchemy / PyMongo

Verification Layers: Python Unit Tests | FastAPI Routes | MySQL Transactions | MongoDB Atlas | Frontend Integration

Conclusion & Future Scope

Engineering Contributions, Current Achievements and Future Deployment

OUR CONCLUSION

Beyond inventory.
Towards accountability.

LabTrack connects hardware availability, requisitions, approvals, custody and demand visibility.

ENGINEERING CONTRIBUTIONS

01. Project-Linked Workflows
Hardware requirements directly linked to academic projects, eliminating rogue checkouts and establishing traceability.

02. Transactional Inventory
MySQL-backed stock, custody and immutable audit records enforcing zero negative inventory and strict ACID integrity.

03. SQL + NoSQL Analytics
Structured multi-table reporting combined with MongoDB document-oriented aggregations for unmet demand forecasting.

THE ROAD TO SIT DEPLOYMENT

Three-stage evolutionary progression from working prototype to campus standard.

1 **STAGE 01: CURRENT — Working Engineering Prototype** IMPLEMENTED

- Functional application and resilient database foundation
- Project-linked inventory, role-based workflows and requisition model
- Advanced relational queries (MySQL) and document analytics (MongoDB Atlas)

2 **STAGE 02: NEXT — SIT R&D Department Pilot** PROPOSED

- Reconcile and onboard real laboratory stock and component catalogs
- Validate custody handovers directly with faculty, lab staff and students
- Establish standard operational procedures (SOPs) and feedback loops

3 **STAGE 03: FUTURE — Institutional Expansion** FUTURE VISION

- Extend deployment across additional SIT engineering departments and laboratories
- Introduce QR/RFID-supported physical custody verification and scanning
- Develop demand-informed procurement analytics to guide institutional purchasing

“Built from a real departmental problem. **Designed for real departmental use.**”

Final Submission Checklist

Project Deliverables, Documentation and Team Contributions

Submissions & Documentation

✓ **01. Report Format Guidelines** ALIGNED
Strict compliance with official DCDS report template

✓ **02. Final Report Moodle Upload** UPLOADED
PDF upload prior to scheduled ESE examination slot

✓ **03. Presentation PPTX Submission** SUBMITTED
13-slide deck conforming to official template boundaries

Readiness & Artifacts

✓ **04. Readable Artifacts & Queries**
High-contrast ER diagrams, relational schema, and SQL

✓ **05. Individual Contribution Split**
Ishaan: Systems & Backend; Raghu: Frontend & UI

✓ **06. 10-Minute Timebox**
Presentation)

References & Q&A

Technical References and Questions from the Evaluation Panel

REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., New York, NY: McGraw-Hill Education, 2020.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Boston, MA: Pearson, 2017.

[3] M. Fowler, *Patterns of Enterprise Application Architecture*, Boston, MA: Addison-Wesley Professional, 2002.

[4] Oracle Corporation, *MySQL 8.4 Reference Manual*, Oracle Corp., 2026.

[5] MongoDB Inc., *MongoDB Atlas Database & Aggregation Framework Documentation*, 2026.

[6] SQLAlchemy Authors, *SQLAlchemy 2.0 Unified Documentation & Expression Language*, 2026.

[7] S. Ramirez, *FastAPI: High Performance Asynchronous Web Framework for Python*, 2026.

Questions & Discussion

Thank you for your time and evaluation.

♦ ♦ ♦

PROJECT

LabTrack: Hardware Availability & Tracking System

INSTITUTION

Department of AI&ML • SIT Pune (2026–27)

♦ ♦ ♦

EXAMINER EVALUATION

Open for architectural walkthrough, schema defense, and live query demonstration.